252-0027

Einführung in die Programmierung

Übungen

Woche 7: Hoare-Logik, Problem Solving

Joran Bühler
Departement Informatik
ETH Zürich

Bonus

Implementieren Sie die Methode `countAssimilated` in der Klasse `Matrix`, welche eine $m \times n$-Matrix A (mit $m > 2, n > 2$) von `int`-Werten als Input akzeptiert und einen `int`-Wert zurückgibt. Für alle Elemente $a_{i,j}$ von A gilt $a_{i,j} \neq 0$.

Diese Methode soll die Anzahl der *an ihre Umgebung angepassten* Elemente in der Input-Matrix A berechnen und zurückgeben. Ein Element $a_{i,j}$ von A gilt als angepasst, wenn die Summe seiner direkten Nachbarn (vertikal, horizontal und diagonal, soweit in der Matrix A definiert) ohne Rest durch $a_{i,j}$ teilbar ist. Das heisst, dass ein Element $a_{i,j}$ genau dann angepasst ist, wenn

$$(a_{i-1,j-1} + a_{i-1,j} + a_{i-1,j+1} + a_{i,j-1} + a_{i,j+1} + a_{i+1,j-1} + a_{i+1,j} + a_{i+1,j+1}) \% a_{i,j} == 0$$

gilt. Wenn das Element $a_{x,y}$ nicht existiert (da $x < 0$ oder $x \geq m, y < 0, y \geq n$), dann wird der Wert in der Berechnung durch 0 ersetzt.

Sie können davon ausgehen, dass der Parameter `matrix` nie den Wert `null` hat und nie auf eine $m \times n$ Matrix mit $m \leq 2$ oder $n \leq 2$ verweist. Sie können auch davon ausgehen, dass die Summe der Nachbarn sich ohne Overflow oder Underflow berechnen lässt.

Hier sind ein paar Beispielaufrufe der Methode sowie das erwartete Ergebnis (im "test"-Ordner finden Sie die entsprechenden JUnit-Tests):

1. `countAssimilated([[10, 10, 10], [10, 10, 10], [10, 10, 10]])`: 9
2. `countAssimilated([[5, 10, 3], [6, 9, 6], [3, 3, 15]])`: 4
3. `countAssimilated([[4, 7, 13], [-2, -12, 32], [20, 15, -8], [17, 3, 1111]])`: 3

Two approaches

```
1 2 3        0 0 0 0 0
4 5 6   →    0 1 2 3 0
             0 4 5 6 0
             0 0 0 0 0
```

```
int[][] padded = new int[rows + 2][cols + 2];

for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        padded[i + 1][j + 1] = matrix[i][j];
    }
}

return padded;
```

```
for (int row = 0; row < rows; row++) {
    for (int col = 0; col < cols; col++) {
        int sum = 0;

        for (int i = row - 1; i <= row + 1; i++) {
            for (int j = col - 1; j <= col + 1; j++) {
                if (i >= 0 && i < rows && j >= 0 && j < cols)
                    if (i != row || j != col) {
                        sum += matrix[i][j];
                    }
                }
            }
        }

        if (sum % matrix[row][col] == 0) {
            count++;
        }
    }
}
```

Weakest Precondition II

Stärkere und schwächere Aussagen

- Wir können *stärkere* und *schwächere* Aussagen unterscheiden
  - Wenn $P_1 \Longrightarrow P_2$ gilt, dann ist P_1 stärker als P_2 (und P_2 schwächer als P_1)
  - Falls P_1 wahr ist, ist P_2 auch wahr.
  - Bsp: P_1: $x > 4$ (Stärker) P_2: $x > 0$ (Schwächer)
- **Starke Aussage:** Eine Aussage oder Voraussetzung, die **mehr Einschränkungen** auferlegt. Sie gilt in weniger Fällen.
- **Schwache Aussage**: Eine Aussage oder Voraussetzung, die **weniger Einschränkungen** auferlegt. Sie ist allgemeiner.

Preconditions / Postconditions

- Die **schwächste Precondition** ist die schwächste Precondition, für die noch die Postcondition erfüllt ist.
  - «Was dürfen wir der Funktion für Werte geben, damit wir einen erwünschten Output haben?»
- Die **stärkste Postcondition** ist die stärkste Postcondition, welche noch aus der Precondition folgt.
  - «Was kann alles als Resultat kommen?»

If-Anweisungen und Aussagen

- **Ziel:** Regel für Tripel {P} if (b) S_1 else S_2 {Q}

- **Beobachtungen**
  - Ausführung von S_1 wenn b hält
  - Ausführung von S_2 wenn ¬b hält
  - P hält in beiden Fällen vor der Ausführung
  - Q muss in beiden Fällen nachher gelten

```
{P}
  if (b) {
    S1;
  } else {
    S2;
  }
{Q}
```

Hoare-Logik-Regel für if-Anweisungen

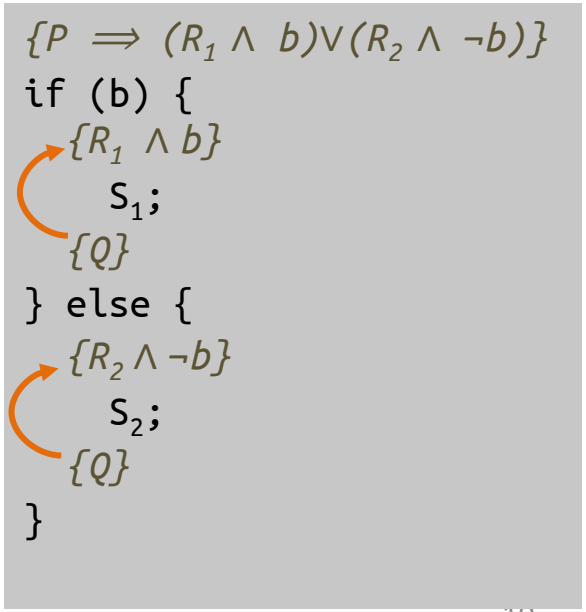
- Das Tripel $\{P\}\ \texttt{if (b)}\ S_1\ \texttt{else}\ S_2\ \{Q\}$ ist gültig, genau dann wenn
  1. $\{P \wedge b\}\ S_1\ \{Q\}$ gültig ist
  2. $\{P \wedge \neg b\}\ S_2\ \{Q\}$ gültig ist

Vorgehen für if-Anweisungen

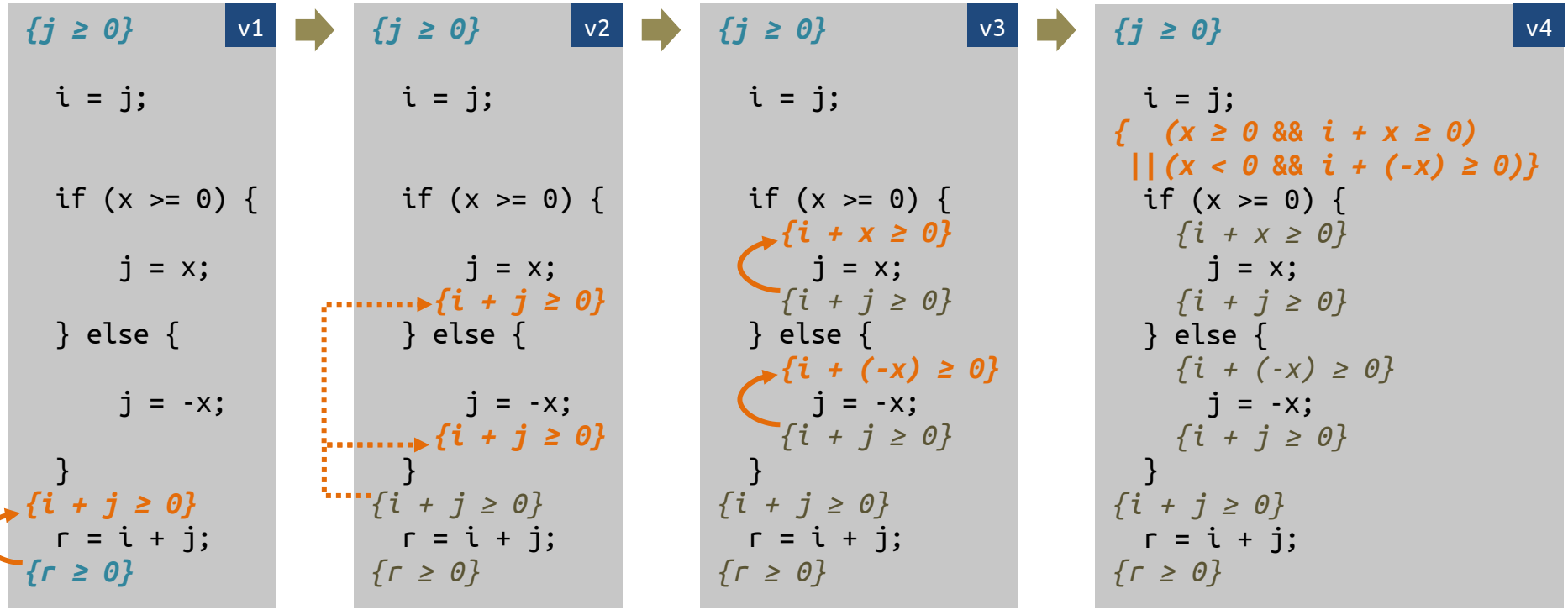
- **Situation:** Entscheide, ob $\{P\}$ `if (b)` S_1 `else` S_2 $\{Q\}$ gültig ist

- **Empfohlenes Vorgehen:**
  1. Wende bekannte Regeln wie rechts gezeigt an (auf S_1 und S_2)
  2. Zeige notwendige Implikation

     $P \Longrightarrow (R_1 \wedge b) \vee (R_2 \wedge \neg b)$



Beispiel Schritt für Schritt



v4

v5

```
{j ≥ 0}
{  (x ≥ 0 && j + x ≥ 0)
|| (x < 0 && j + (-x) ≥ 0)}
  i = j;
{  (x ≥ 0 && i + x ≥ 0)
 ||(x < 0 && i + (-x) ≥ 0) }
  if (x >= 0) {
    {i + x ≥ 0}
      j = x;
    {i + j ≥ 0}
  } else {
    {i + (-x) ≥ 0}
      j = -x;
    {i + j ≥ 0}
  }
{i + j ≥ 0}
  r = i + j;
{r ≥ 0}
```

Dann noch zu zeigen:

$$(j \geq 0) \Rightarrow ((x \geq 0 \;\&\&\; j + x \geq 0) \;||\; (x < 0 \;\&\&\; j + (-x) \geq 0))$$

Fallunterscheidung (zwei Konjunkte):

1. Wenn $j \geq 0$ und $x \geq 0$, dann $j + x \geq 0$ ✓
2. Wenn $j \geq 0$ und $x < 0$, dann $j + (-x) \geq 0$ ✓

Hinweis!

- **Ihr könnt nutzen:**

```
(j ≥ 0) ⟹ ((x ≥ 0 ⟹ j + x ≥ 0) && (x < 0 ⟹ j + (-x) ≥ 0))
(j ≥ 0) ⟹ ((x ≥ 0 && j + x ≥ 0) || (x < 0 && j + (-x) ≥ 0))
```

- **Aber nur hier! Weil bei if / else zwei Fälle unterscheiden, die zusammen alle Möglichkeiten abdecken.**

- **Das gilt sonst nicht.**

Beispiel 1: Gültiges Tripel

Zu zeigen: Gültigkeit von $\{true\}$ `if (x < 0) { y = -x; } else { y = x;}` $\{y \geq 0\}$

1. Regeln anwenden:

```
if (x < 0) {
  {true ∧ x < 0}
  {-x ≥ 0}
    y = -x;
  {y ≥ 0}
} else {
  {true ∧ ¬(x < 0)}
  {x ≥ 0}
    y = x;
  {y ≥ 0}
}
```

2. Implikationen zeigen:
   1. $(x < 0) \implies (-x \geq 0)$ ✓
   2. $(x \geq 0) \implies (x \geq 0)$ ✓

Beispiel 2: Ungültiges Tripel

Geändert

Zu zeigen: Gültigkeit von $\{true\}$ `if (x < 0) { y = -x; } else { y = x;}` $\{y \geq 0\}$

1. Regeln anwenden:

```
if (x < 0) {
  {true ∧ x < 0}
  {-x ≥ 0}
    y = -x;
  {y ≥ 0}
} else {
  {true ∧ ¬(x < 0)}
  {x ≥ 0}
    y = x;
  {y ≥ 0}
}
```

Entsprechende Unterschiede

2. Implikationen zeigen:
   1. $(x < 0) \Longrightarrow (-x \geq 0)$ ✓
   2. $(x \geq 0) \Longrightarrow (x \geq 0)$ ✗

Hält nicht mehr (Gegenbeispiel: $x == 0$)

Beispiel 3: Ohne else-Zweig

Zu zeigen: Gültigkeit von $\{x == y \land x \mathrel{!}= 0\}$ `if (a > 1) { a = a * x; }` $\{a \mathrel{!}= y\}$

1. Regeln anwenden:

```
if (a > 1) {
  {x == y ∧ x != 0 ∧ a > 1}
  {a * x ≠ y}
    a = a * x;
  {a ≠ y}
} else {
  {x == y ∧ x != 0 ∧ ¬(a > 1)}
  {a ≠ y}
}
```

Leerer else-Block entspricht fehlendem else-Block

2. Implikationen zeigen:

1. ✓ $(x == y \land x \mathrel{!}= 0 \land a > 1) \Longrightarrow (a * x \mathrel{!}= y)$
2. ✗ $(x == y \land x \mathrel{!}= 0 \land a \leq 1) \Longrightarrow (a \mathrel{!}= y)$

Weakest Precondition mit Verzweigung

Schwächste Vorbedingung - Beispiel

```
{                }

  if (x >= y){
    y = x
  }

{y >= x}
```

Weakest Precondition – Mit Verzweigung

{ }

```
if (x<5) {
        y = x*x;
} else {
        y = x+1;
}
```

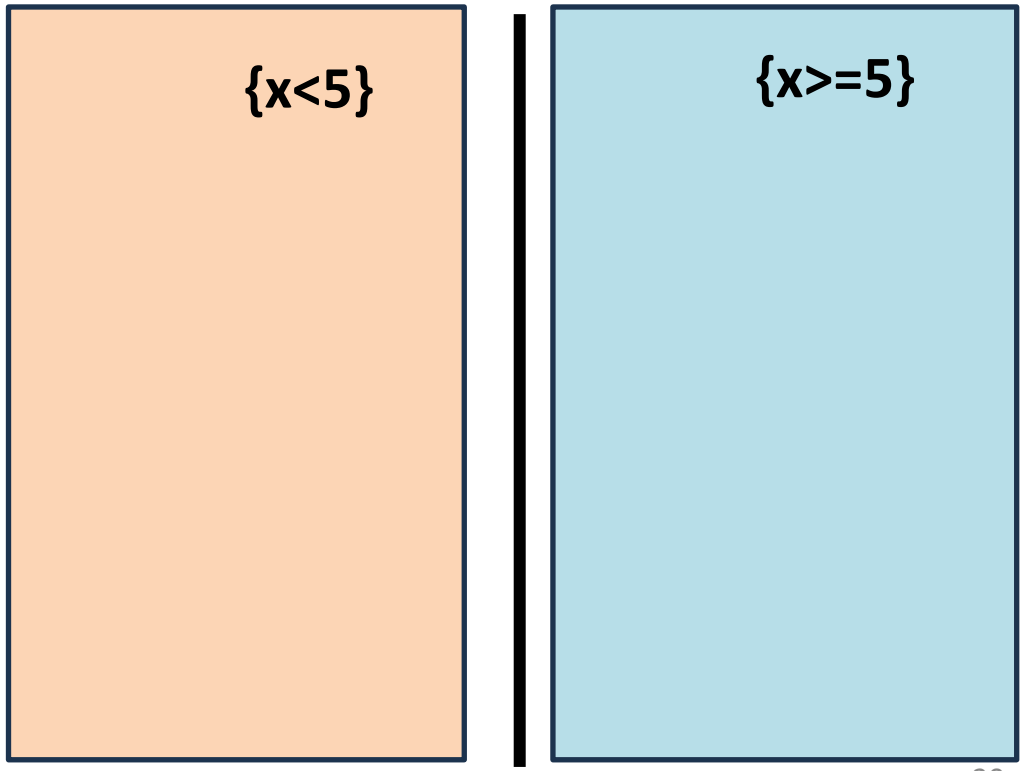
{y>9}

Weakest Precondition – Mit Verzweigung

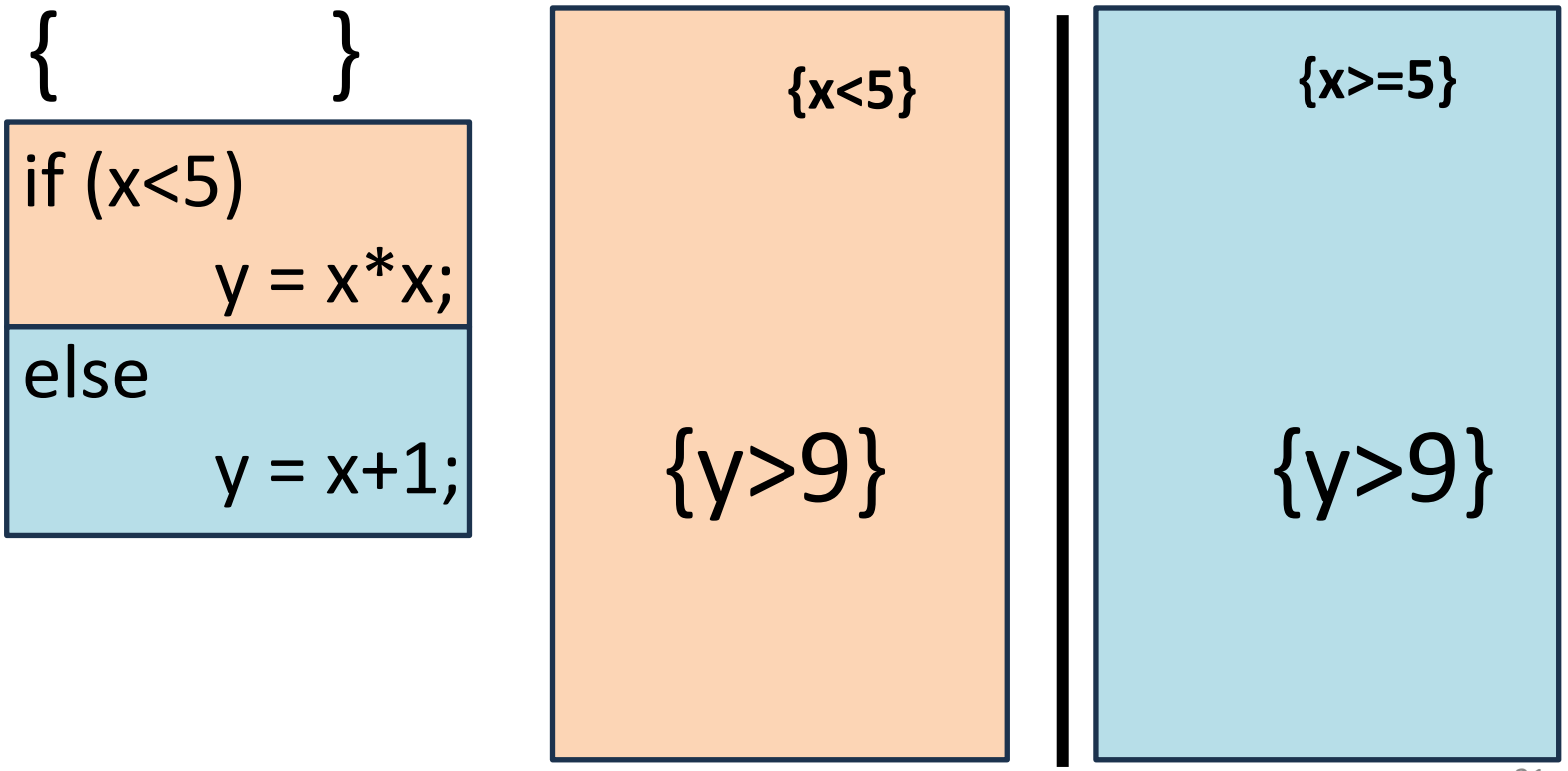
{ }

```
if (x<5)
        y = x*x;
else
        y = x+1;
```

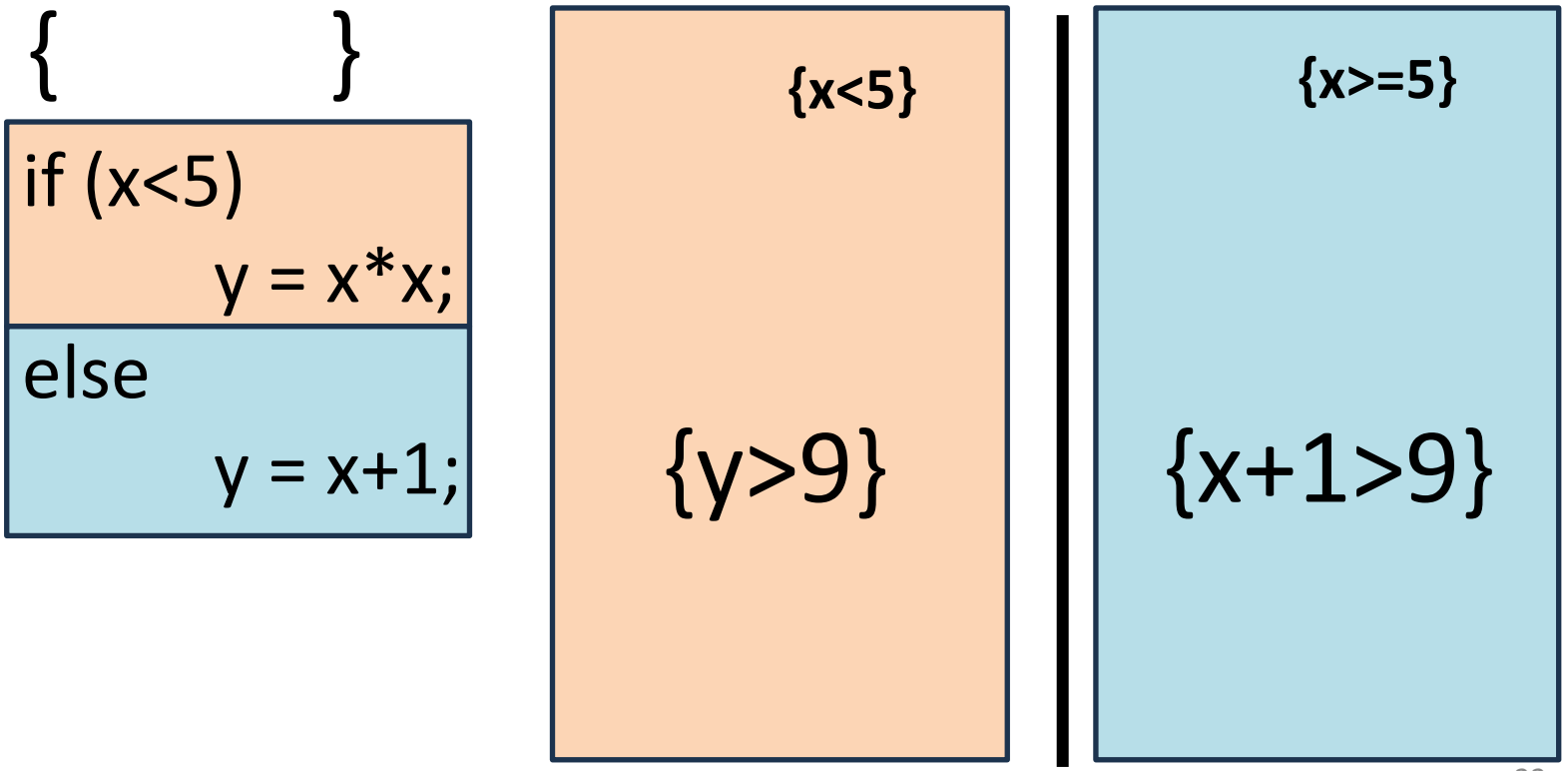
{y>9}



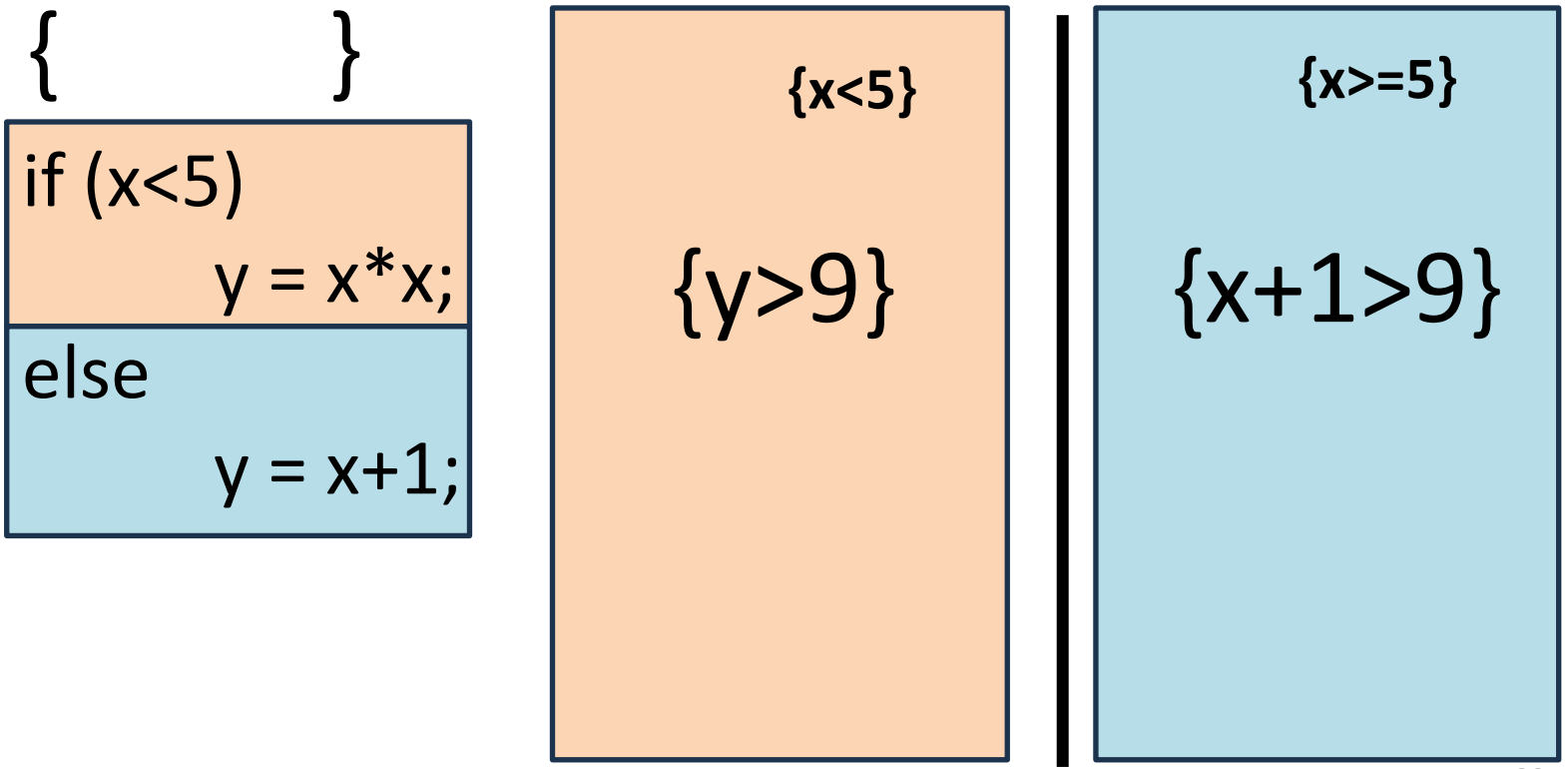
Weakest Precondition – Mit Verzweigung



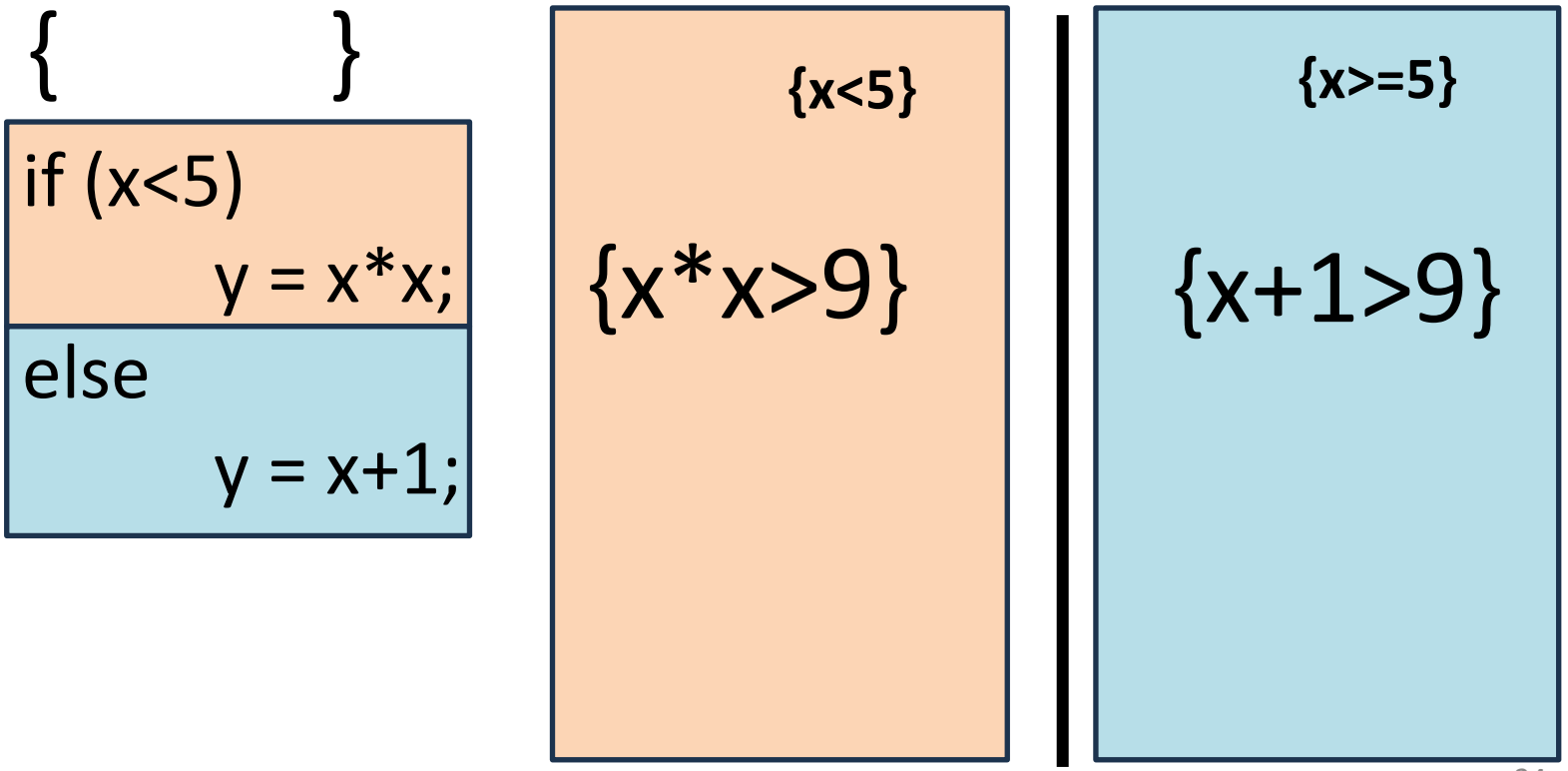
Weakest Precondition – Mit Verzweigung



Weakest Precondition – Mit Verzweigung



Weakest Precondition – Mit Verzweigung



{(**x<5** && x*x>9) || (**x>=5** && x+1>9)}

{ }

```
if (x<5)
        y = x*x;
else
        y = x+1;
```

{x<5}

{x*x>9}

||

{x>=5}

{x+1>9}

Weakest Precondition - Prüfungsbeispiel

```
{              }
 if (x > y)
        z = x – y;
 else
        z = y – x;

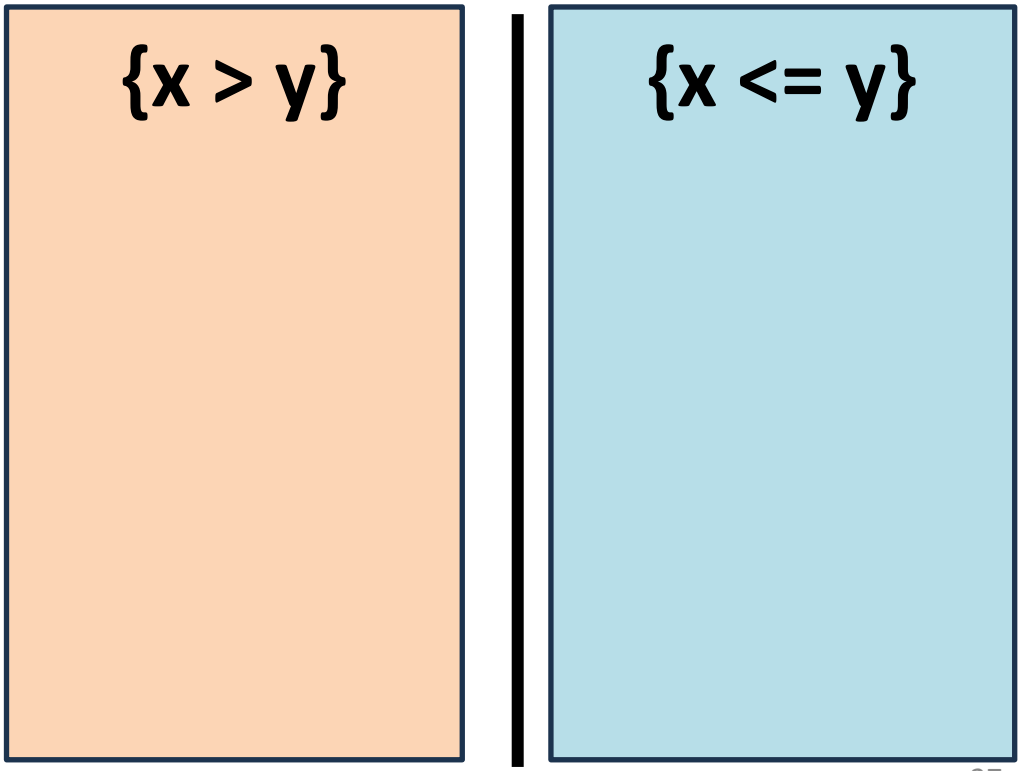
{  z > 0  }
```

Weakest Precondition - Prüfungsbeispiel

{ }

```
if (x > y)
        z = x – y;
else
        z = y – x;
```

{ z > 0 }

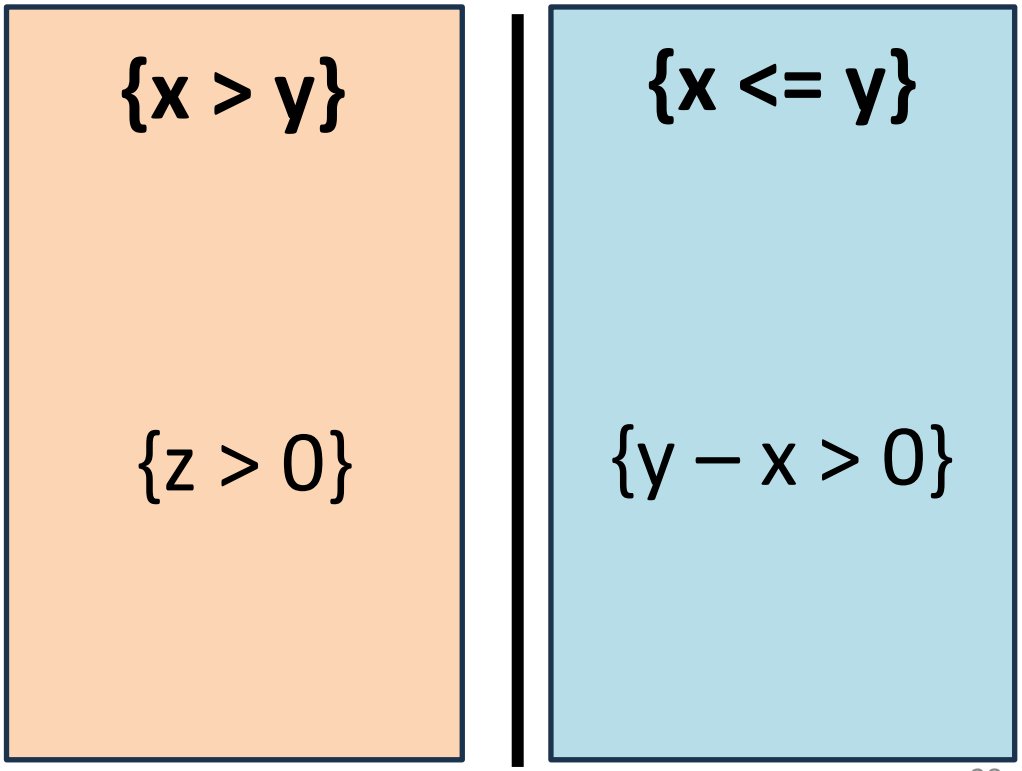


Weakest Precondition - Prüfungsbeispiel

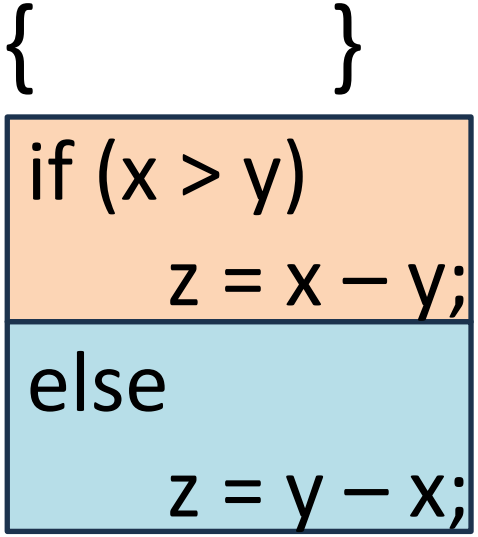
{ }

```
if (x > y)
        z = x – y;
else
        z = y – x;
```

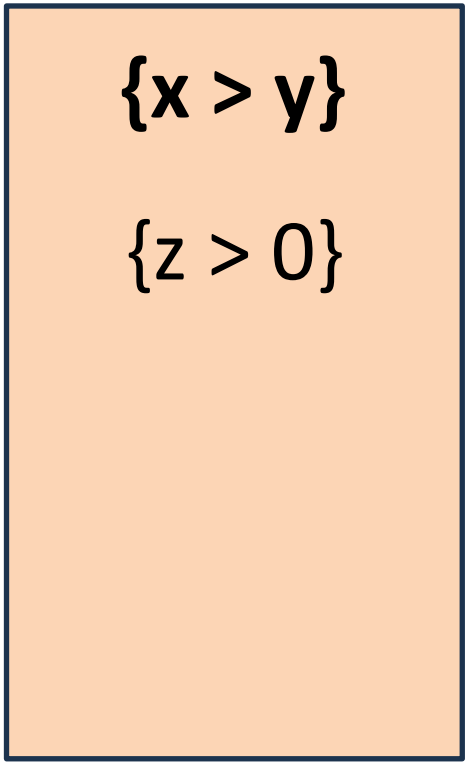
{ z > 0 }

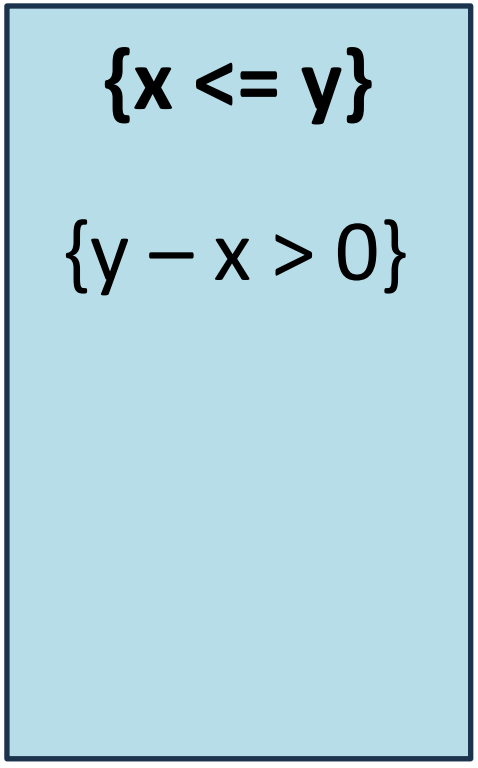


Weakest Precondition - Prüfungsbeispiel



{ z > 0 }



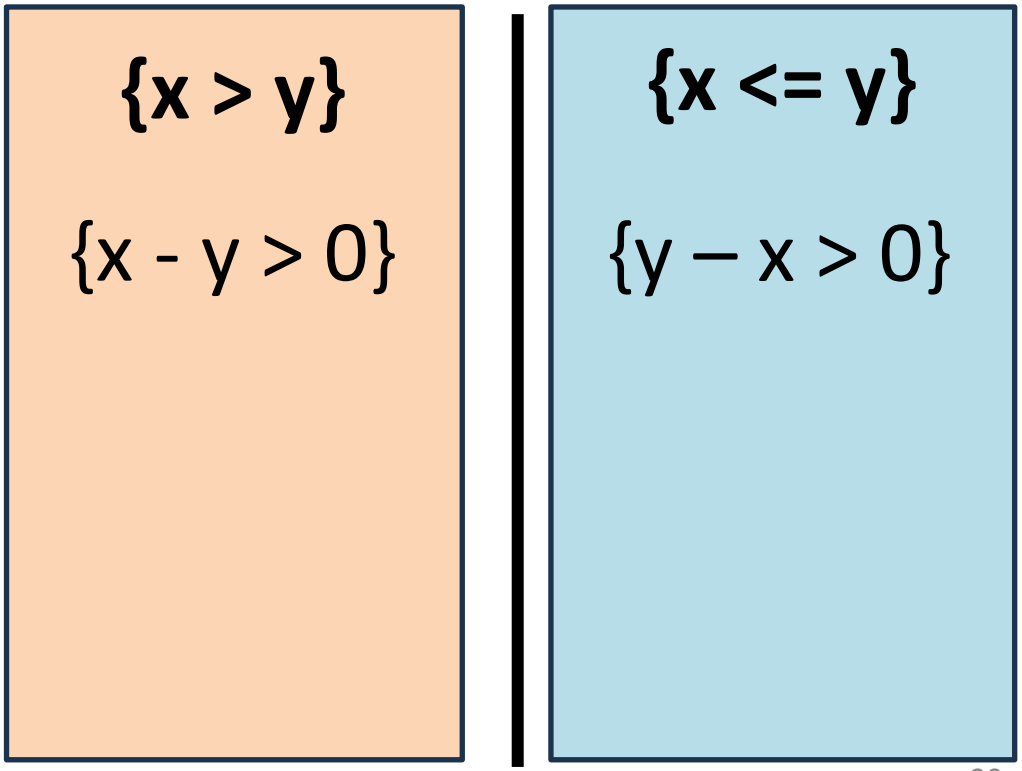


Weakest Precondition - Prüfungsbeispiel

{ }

```
if (x > y)
        z = x – y;
else
        z = y – x;
```

{ z > 0 }



Weakest Precondition - Prüfungsbeispiel

{ }

```
if (x > y)
        z = x – y;
else
        z = y – x;
```

{ z > 0 }

{x > y}

{x > y}

{x <= y}

{y – x > 0}

{((x > y) && (x > y)) || ((x <= y) &&(y > x))}

{ }

```
if (x > y)
        z = x – y;
else
        z = y – x;
```

{ z > 0 }

{x > y}

{x > y}

{x <= y}

{y > x}

Hoare Triple

Hoare Tripel – Prüfungsbeispiel HS18

```
{ b > c }
 if (x > b) {
     a = x;
} else {
     a = b;
}
{ a > c }
```

Hoare Tripel – Prüfungsbeispiel HS18

```
{ x > 0 }
    y = x * x;
    z = y / 2;
{ z > 0 }
```

Hoare Logik FAQ

- Müssen wir unsere Ausdrücke vereinfachen?
  - Solange die Lösung **logisch äquivalent** ist, ist sie korrekt.

- Müssen wir unsere Herleitung zeigen?
  - Solange dies **nicht explizit gefragt** wird, ist dies nicht nötig.

- Müssen Gegenbeispiele alle Variablen enthalten, wenn nur eine davon relevant ist?
  - Wenn z.B. y != 0 immer bedeutet, das aus einer Precondition nicht die Postcondition folgt, dann ist dies ausreichend.

Nachbesprechung

Aufgabe 1: Präfixkonstruktion

Gegeben seien zwei Strings `s` und `t` und ein Integer `n` mit $n \geq 0$. Schreiben Sie ein Programm, das zurückgibt, ob `s` eine Konkatenation von maximal `n` vielen Präfixen von `t` ist.
Beispiele:

- `s = "abcababc"`, `t = "abc"`, `n = 4`: Das Programm sollte `true` zurückgeben, da `"abc"` und `"ab"` Präfixe von `t` sind und `s` eine Konkatenation von `"abc"`, `"ab"`, `"abc"` ist.
- `s = "abcbcabc"`, `t = "abc"`, `n = 4`: Das Programm sollte `false` zurückgeben, da `"bc"` kein Präfix von `t` ist.
- `s = "abab"`, `t = "abac"`, `n = 2`: Das Programm sollte `true` zurückgeben, da `"ab"` ein Präfix von `t` ist und `s` eine Konkatenation von `"ab"`, `"ab"` ist.

Implementieren Sie die Methode `isPrefixConstruction(String s, String t, int n)` in der Klasse `PrefixConstruction`. Die Methode hat drei Argumente: die beiden Strings `s` und `t` und der Integer `n`. Sie dürfen davon ausgehen, dass der Integer grösser oder gleich 0 ist. In der Datei "PrefixConstructionTest.java" finden Sie Tests.

Tipp: Lösen Sie die Aufgabe rekursiv.

Aufgabe 1: Präfixkonstruktion

- **Greedy approach funktioniert nicht immer!**
- **Lösung: Rekursion mit backtracking**

```
s="abab", t="abac", n=2
```

Example Solution

```java
public class PrefixConstruction {

    public static boolean isPrefixConstruction(String s, String t, int n) {
        if (s.isEmpty()) return true;
        if (n == 0) return false;

        for (int i = 1; i <= t.length(); i++) {
            String prefix = t.substring(0, i);

            if (s.startsWith(prefix)) {
                String rest = s.substring(i);

                if (isPrefixConstruction(rest, t, n - 1)) {
                    return true;
                }
            }
        }

        return false;
    }
}
```

Kahoot

- https://create.kahoot.it/share/u7-eprog-recursion-wpc-and-hoare-tripel/26632932-a421-44f1-a64a-7107e1818286

Permutation Check

Beispielsaufgabe

Gegeben sind die Integer Arrays `s` und `t`, überprüfe, ob `t` eine Permutation von `s` ist.

Lösung 1: Generiere alle Permutationen von `s` und schaue ob `t` eine davon ist.

Beispielsaufgabe

Gegeben sind die Integer Arrays s und t, überprüfe, ob t eine Permutation von s ist.

Lösung 1: Generiere alle Permutationen von s und schaue ob t eine davon ist.

```
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
        at TestingStuff/module.Permutation.permute(Permutation.java:16)
        at TestingStuff/module.Permutation.permute(Permutation.java:23)
        at TestingStuff/module.Permutation.permute(Permutation.java:23)
        at TestingStuff/module.Permutation.permute(Permutation.java:23)
        at TestingStuff/module.Permutation.permute(Permutation.java:23)
        at TestingStuff/module.Permutation.permute(Permutation.java:23)
```

Out of Memory?

- Java erhält für das Ausführen eines Programms eine **beschränkte** Menge Arbeitsspeicher (Memory).

- In der vorherigen Lösung generieren wir **O(n!) Permutation**, welche alle Speicher benötigen.

- Irgendwann hat es **nicht mehr genug Speicher** und das Programm terminiert mit einem Fehler (Error).

Beispielsaufgabe

Gegeben sind die Integer Arrays s und t, überprüfe, ob t eine Permutation von s ist.

Lösung 2: Generiere alle Permutationen von s und schaue ob t eine davon ist, während wir die Permutationen generieren.



Beispielsaufgabe

Gegeben sind die Integer Arrays `s` und `t`, überprüfe, ob `t` eine Permutation von `s` ist.

Lösung 3: Prüfe ob `s` und `t` gleich lang sind und ob sie die Gleichen Elemente enthalten.

Vorbesprechung

Aufgabe 1: Dreiecksmatrix



Die Klasse `Triangle` erlaubt die Darstellung von $Z \times S$ Dreiecksmatrizen (von `int` Werten). Z und S sind immer strikt grösser als 1 (d.h., > 1). Eine $Z \times S$ Dreiecksmatrize hat Z Zeilen $X_0, X_1, ..., X_{Z-1}$, wobei Zeile X_i genau $(i * (S-1)/(Z-1)) + 1$ viele Elemente hat. Dieser Ausdruck wird nach den Regeln für `int` Ausdrücke in Java ausgewertet. Für eine Dreiecksmatrix `D` ist $D_{i,j}$ das $(j+1)$-te Element in der $(i+1)$-ten Zeile. $D_{0,0}$ ist das erste Element in der ersten Zeile [die immer genau 1 Element hat]. Abbildung 1 zeigt Beispiele von Dreiecksmatrizen. Beachten Sie, dass es möglich ist, dass zwei (aufeinanderfolgende) Zeilen die selbe Anzahl Elemente haben.

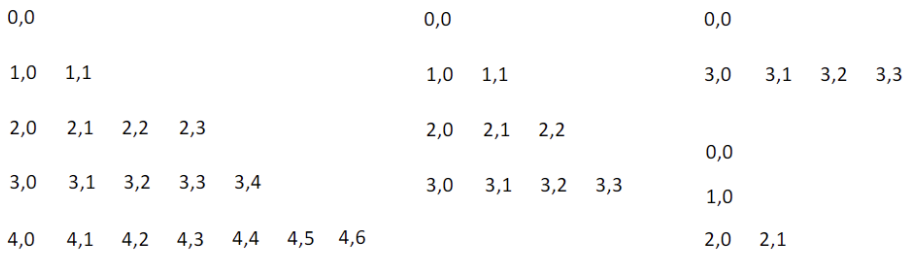


Abbildung 1: Beispiele von 5×7, 4×4, 2×4 und 3×2 Dreiecksmatrizen.

Aufgabe 1: Dreiecksmatrix

In der Datei `Triangle.java` finden Sie die Klasse `Triangle` mit einem Konstruktor `Triangle(int z, int s)`, der eine $z \times s$ Dreiecksmatrix erstellt. Dieser Konstruktor setzt die Werte aller Elemente auf 0. Vervollständigen Sie diese Klasse, so dass die folgenden Methoden unterstützt werden:

1. `int get(int i, int j)` gibt das Element $D_{i,j}$ zurück.
2. `void put(int i, int j, int value)` setzt das Element $D_{i,j}$ auf den Wert `value`.
3. `int[] linear()` liefert die Elemente in der kanonischen Reihenfolge (die Elemente jeder Zeile mit steigendem Index, und die Zeilen in steigender Reihenfolge).
4. `void init(int[] data)` ersetzt die Elemente von `D` durch die Werte in `data`. Sie dürfen annehmen, dass `data` genauso viele Elemente hat wie `D`. Die Methode setzt die Elemente von `D`, so dass die Folge `D.init(data); int[] y = D.linear();` in einen Array `y` resultiert für den `Arrays.equals(y, data)` den Wert `true` ergibt.
5. `void add(Triangle t)` Ein Aufruf `D.add(t)` addiert zu jedem Element $D_{i,j}$ den Wert von $t_{i,j}$, falls $t_{i,j}$ existiert. Falls $t_{i,j}$ nicht existiert, dann bleibt $D_{i,j}$ unverändert.

Tests finden Sie in der Datei "TriangleTest.java". Die Datei "TriangleGradingTest.java" enthält die Tests, welche wir bei der Prüfung für die Korrektur verwendet haben. Wir empfehlen, diese Tests erst zu verwenden, wenn Sie denken, dass Ihre Lösung korrekt ist, damit Sie sehen können, wie Sie bei einer Prüfung abgeschnitten hätten.

Aufgabe 2: Hoare Tripel



Hinweis: Diese Aufgabe wird als Moodle-Quiz bearbeitet.

Aufgabe 3: Weakest Precondition



Hinweis: Diese Aufgabe wird als Moodle-Quiz bearbeitet.

Aufgabe 4: Blackbox Testing



Im letzten Übungsblatt haben Sie Testautomatisierung mit JUnit kennengelernt. In dieser Aufgabe sollen Sie nun Tests für eine Methode schreiben, deren Implementierung Sie nicht kennen. Dadurch werden Sie weniger durch möglicherweise falsche Annahmen beeinflusst, die bei einer Implementierung getroffen wurden. Sie müssen sich also überlegen, wie sich *jede* fehlerfreie Implementierung verhalten muss. Diesen Ansatz nennt man auch Black-Box Testing da die Details der Implementation verdeckt sind.

In Ihrem "U06"-Projekt befindet sich eine "blackbox.jar"-Datei, welche eine kompilierte Klasse `BlackBox` enthält. Den Code dieser Klasse können Sie nicht sehen, aber sie enthält eine Methode `void rotateArray(int[] values, int steps)`, welche Sie aus einer eigenen Klasse oder einem Unit-Test aufrufen können. Diese Methode "rotiert" ein `int`-Array um eine gegebene Anzahl Schritte.

Vereinfacht macht die Methode `rotateArray()` Folgendes: Eine Rotation mit `steps=1` bedeutet, dass alle Elemente des Arrays um eine Position nach rechts verschoben werden. Das letzte Element wird dabei zum ersten. Mit `steps=2` wird alles um zwei Positionen nach rechts rotiert, usw. Eine Rotation nach links kann mit einer negativen Zahl für `steps` erreicht werden. Das folgende Beispiel ist der erste, einfache Test, den Sie in der Datei "BlackBoxTest.java" finden:

```
int[] values = new int[] { 1, 2 };
int[] expected = new int[] { 2, 1 };
BlackBox.rotateArray(values, 1);
assertArrayEquals(expected, values);
```

Aufgabe 5: Levels



Implementieren Sie die Methode `sumLevel(int[][] matrix, int level)` in der Klasse `Levels`, die für eine quadratische Matrix (Parameter `matrix`) und eine angegebene „Level"-Nummer (Parameter `level`) die Summe der Elemente dieses Levels berechnet.
Ein Level ist definiert durch die Schichten der Matrix, die von innen nach aussen gezählt werden. Level 0 entspricht dem Zentrum der Matrix. Höhere Level beschreiben jeweils die nächsten, äusseren Ränder der Matrix.

In Abbildung 2 finden Sie zwei Beispiele für Matrizen und ihre Levels. In Abbildung 2a beispielsweise besteht das innerste Level, Level 0, lediglich aus einem Element. Abbildung 2b zeigt eine Matrix mit drei Levels: Der äussere Rand der Matrix ist Level 2, der mittlere Rand Level 1 und Level 0 besteht aus den innersten vier Elementen.

Aufgabe 5: Levels

1 Level: 1	2 Level: 1	3 Level: 1
4 Level: 1	5 Level: 0	6 Level: 1
7 Level: 1	8 Level: 1	9 Level: 1

(a) 3 × 3-Matrix

10 Level: 2	20 Level: 2	30 Level: 2	40 Level: 2	50 Level: 2	60 Level: 2
15 Level: 2	25 Level: 1	35 Level: 1	45 Level: 1	55 Level: 1	65 Level: 2
70 Level: 2	80 Level: 1	90 Level: 0	100 Level: 0	110 Level: 1	120 Level: 2
5 Level: 2	10 Level: 1	15 Level: 0	20 Level: 0	25 Level: 1	30 Level: 2
1 Level: 2	2 Level: 1	3 Level: 1	4 Level: 1	5 Level: 1	6 Level: 2
99 Level: 2	88 Level: 2	77 Level: 2	66 Level: 2	55 Level: 2	44 Level: 2

(b) 6 × 6-Matrix

Abbildung 2: Beispiele für gültige Matrizen und ihre Levels

Timed Bonus

- **Release: 03.11.2026 17:00 – 19:00**
- **Simuliert die Prüfungskonditionen**

Prüfungsaufgabe Methodenverständnis FS 2021

Gegeben sei eine Java Methode `test()`, die mit verschiedenen Argumenten für ein Exemplar `s` der Klasse `Suspicious` aufgerufen wird.

```
class Suspicious {

    int i = 0;

    String test(int i) {
        if (i <= 0) {
            return "Z";
        }
        if (i < 3) {
            i = i - 1;
            String result = "T" + test(i) + "T";
            i = 0;
            return result;
        } else if (i < 6) {
            return "S" + test(i+1) + "S";
        } else {
            return "X" + test(i/8);
        }
```

```
Suspicious s = new Suspicious();
```

1. `System.out.println(s.test(1));`

   TZT

2. `System.out.println(s.test(3));`

   SSSXZSSS

3. `System.out.println(s.test(9));`

   XTZT

Bitte geben Sie für jeden dieser Aufrufe das Resultat an (entweder dem String, der gedruckt wird, oder schreiben Sie "Laufzeitfehler" (oder "exception") in dem Fall dass eine Exception auftritt (den genauen Fehler bzw. die genaue Exception müssen Sie nicht angeben).

Notfallaufgabe FS 2021

Gegeben sei eine Methode `main` in einer Java Klasse.

```
public static void main(String[] args) {
    /* body */
}
```

Die folgenden Anweisungen sollen als "Body" (Rumpf) anstelle des Kommentars `/* body */` eingefügt werden und den angegebenen Output produzieren. Bestimmen Sie für jede Anweisung die fehlenden Operatoren so, das die Anweisung die gezeigte Ausgabe erzeugt. Mögliche Operatoren sind `+`, `-`, `*`, `/` und `%`.

Wenn es für eine Anweisung mehrere mögliche Lösungen gibt, so genügt eine Lösung. Sollte es keine Lösung geben, so schreiben Sie bitte "nicht möglich".

Bereits existierende Operatoren (oder sonstige Teile der Anweisung) dürfen Sie *nicht* verändern. Auch dürfen Sie Klammern weder hinzufügen noch entfernen.

1. `System.out.println(64 / (8 2) + (15 14) 2) ;`

   `//Output: 18`

2. `System.out.println(((48 8) * (17 3) <= 3 ) || (7 0) == 1);`

   `//Output: true`

3. `System.out.println("test " + 17 17 8 2);`

   `//Output: test 0`